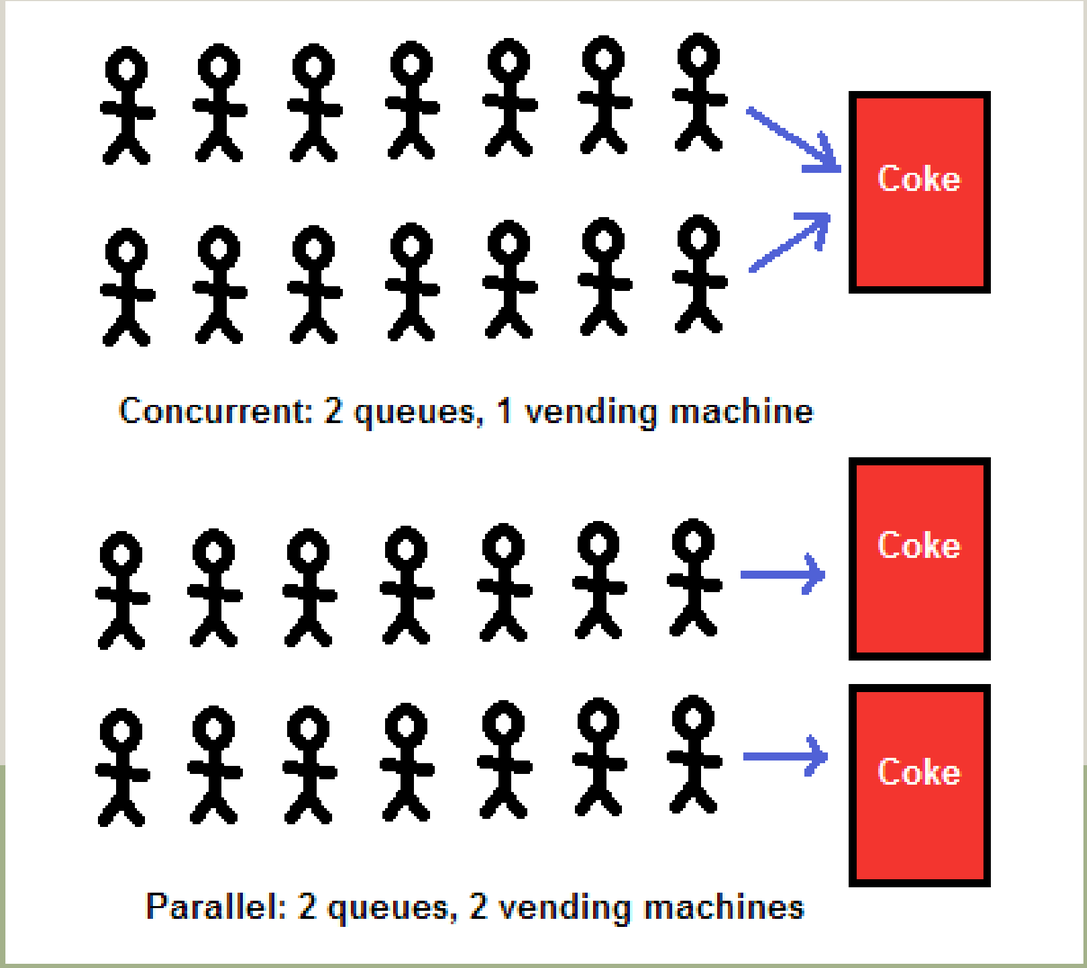
Coding Club, IIT Guwahati

C++ Multithreading features

Session 3 : Synchronization and Condition Variables



Toshit Jain

Recap :)

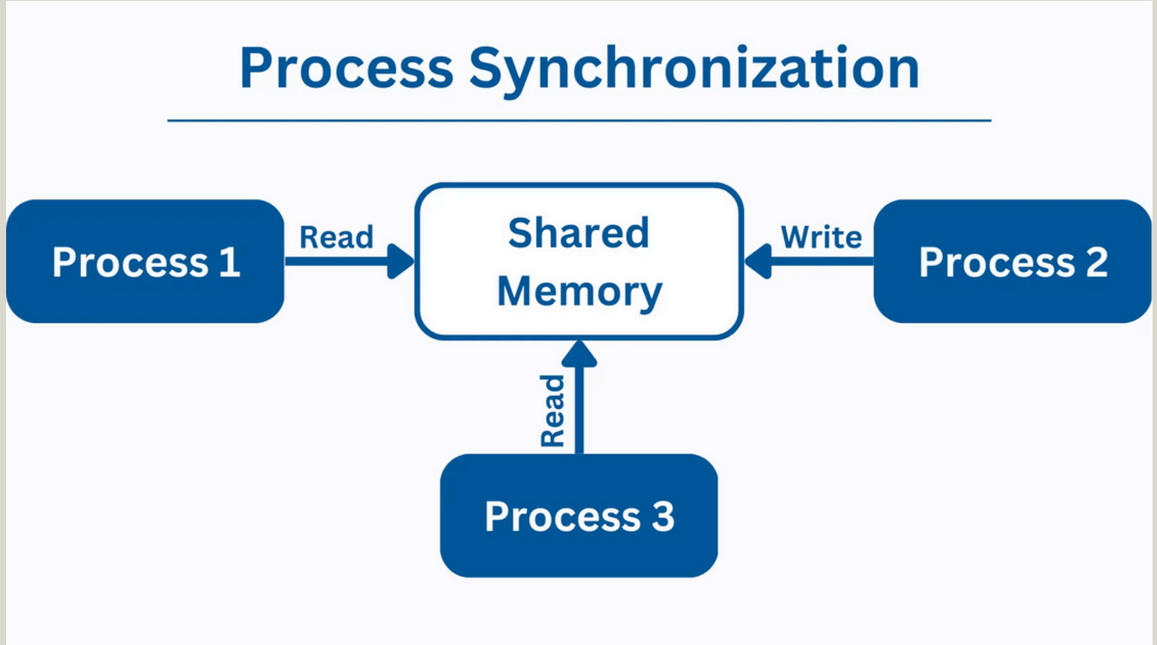
- Last session we studied about protecting our shared data. We get to know about on method of protecting data which is using **std::mutex**.

- We saw the two RAII based classes **std::lock_guard** and **std::unique_lock**. **std::lock_guard** follows strict RAII scheme, locking on construction and unlocking on destruction. However, **std::unique_lock** is a little flexible, not owning the lock at every moment.

- We also made a **prototype** for our **Locking, Unbounded Threadsafe Queue** which used **std::queue** as the underlying storage scheme and **std::mutex** as the concurrency mechanism. We also joined our **pop() & front()** into single **try_pop()** function.

- The **try_pop()** function was overloaded to give a **reference parameter with bool return type** and **std::shared_ptr<T> return type with void parameter**.

Need for synchronization



1. We saw about protecting our shared data. However, many times it is not sufficient.

2. We sometimes want to **synchronize** threads. What is synchronization? Synchronization means forcing a fixed order of execution for the threads. It might be that one thread wants to execute only after another thread has completed a particular event.

3. The need to synchronize operations between threads like this is such a common scenario that the C++ Standard Library provides facilities to handle it, in the form of condition variables and futures

How to tell the “time” to start

1. We want to enforce synchronization. However, a major problem is how would a thread know when the other thread performed the event it was waiting for.

2. One way it could do that is, it could just keep checking a flag in shared data (protected by a mutex) and have the second thread set the flag when it completes the task. This is wasteful on two counts: the thread consumes valuable processing time repeatedly checking the flag, and when the mutex is locked by the waiting thread, it can’t be locked by any other thread.

3. A second option is to have the waiting thread sleep for small periods between the checks using the **std::this_thread::sleep_for()** function. This is an improvement, because the thread doesn’t waste processing time while it’s sleeping, but it’s hard to get the sleep period right. Too short a sleep in between checks and the thread still wastes processing time checking; too long a sleep and the thread will keep on sleeping even when the task it’s waiting for is complete, introducing a delay.

4. The last option is to use language specific features, and C++ does this by giving us access to **std::condition_variable**.

std::condition_variable

main ×

```cpp
#include <iostream>
#include <mutex>
#include <condition_variable>

std::mutex m;
std::queue<int> q;
std::condition_variable cond;

void producer(){
    while(true){
        int val = prepare_data();
        std::lock_guard<std::mutex> lk(m);
        q.push(val);
        cond.notify_one();
    }
}

void consumer(int& val){
    while(true){
        std::unique_lock<std::mutex> lk(m);
        cond.wait(lk, []{return !q.empty();});
        val = q.front();
        q.pop();
    }
}
```

1. **std::condition_variable** is included in the **<condition_variable>** header.

2. On the left, you have a producer thread which prepares data, and once done locks the mutex using **std::lock_guard** to push the data into the queue and call the **std::condition_variable::notify_one()**

3. On the consumer side of things, the mutex is locked by a **std::unique_lock** instead of **std::lock_guard**. Here the condition variable calls **std::condition_variable::wait()** passing the unique lock object and a lambda function. This lambda function checks whether is queue is empty or not and returns true is queue is not empty.

std::condition_variable

1. The implementation of **wait()** then **checks the condition** (by calling the supplied lambda function) and returns if it's satisfied (the lambda function returned true). If the **condition isn't satisfied** (the lambda function returned false), **wait() unlocks the mutex and puts the thread in a blocked or waiting state**. When the condition variable is notified by a call to **notify_one()** from the data-preparation thread, the **thread wakes from its slumber (unblocks it)**, **reacquires the lock on the mutex, and checks the condition again**, returning from wait() with the mutex still locked if the condition has been satisfied. **If the condition hasn't been satisfied, the thread unlocks the mutex and resumes waiting**.

2. This is why you need the std::unique_lock rather than the std::lock_guard, the waiting thread must unlock the mutex while it's waiting and lock it again afterward, and std::lock_guard doesn't provide that flexibility.

3. Instead of lambda function, we can use any **bool returning callable**.

4. When the waiting thread reacquires the mutex and checks the condition, if it isn't in direct response to a notification from another thread, it's called a **spurious wake**. Because the number and frequency of any such spurious wakes are by definition indeterminate, it isn't advisable to use a function with side effects for the condition check.

Blocking, TSF Queue Prototype 2.0

```
void push(T value){
    std::lock_guard<std::mutex> lk(m);
    q.push(value);
    cond.notify_one();
}
```

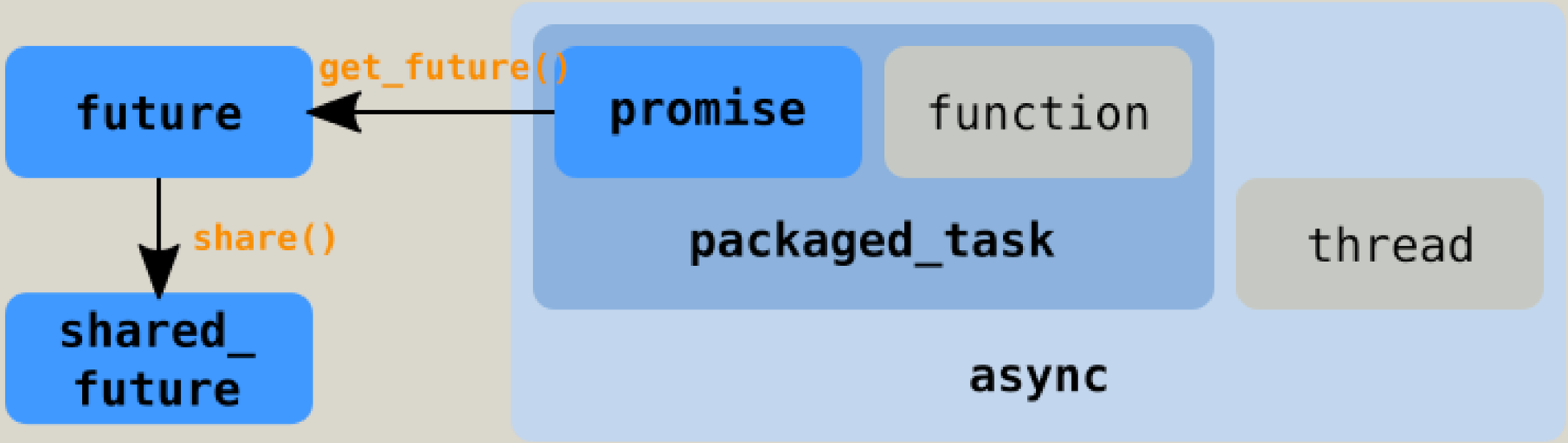
Onto VS Code :)

When new data is ready, the call to notify_one() will trigger one of the threads currently executing wait() to check its condition and thus return from wait() (because you've just added an item to the data_queue). There's no guarantee which thread will be notified or even if there's a thread waiting to be notified; all the processing threads might be still processing data. We also have a notify_all() call.

```
bool wait_and_pop(T& value){
    std::unique_lock<std::mutex> lk(m);
    cond.wait(lk, [this]{
        if(q.empty()) std::cout<<"Waiting."<<'\n';
        return !q.empty(); });
    if(q.empty()) return false;
    value = q.front();
    q.pop();
    return true;
}
```

```
std::shared_ptr<T> wait_and_pop(){
    std::unique_lock<std::mutex> lk(m);
    cond.wait(lk, [this]{
        if(q.empty()) std::cout<<"Waiting."<<'\n';
        return !q.empty(); });
    if(q.empty()) return nullptr;
    std::shared_ptr<T> res(std::make_shared<T>(q.front()));
    q.pop();
    return res;
}
```

std::future, std::async, std::promise & std::packaged_task



What is this image ?? We will come back in just a moment..

std::future and Why ??

1. Inlcuded in the **<future>** header.

2. It can be that a thread is waiting for a specific **one-off** event. Once the event occurs, it can continue processing. Checking that event every time can waste a lot of time. Thus C++ provides us with **std::future** which gives the thread capabilities to do other tasks while periodically checking for the one-off event.

3. We have two types of futures with us. **std::future** which is *unique future* and **std::shared_future** which is *shared future*. For a shared future, when the one-off event finishes, all the objects get ready at the same time.

4. **std::future** are as discussed before, used to return value, something **std::thread** was not able to do.

std::future and std::async

main ×

```
#include <iostream>
#include <future>

int func_with_return(void){
    // do someting
    return 0;
}

int main(){
    std::future<int> fut = std::async(func_with_return);
    do_other_stuff();
    std::cout<<fut.get()<<'\n';
}
```

You use **std::async** to start an asynchronous task for which you don't need the result right away. Rather than giving you back a **std::thread** object to wait on, **std::async** returns a **std::future** object, which will eventually hold the return value of the function. When you need the value, you just call **get()** on the future, and the thread blocks until the future is ready and then returns the value.

std::async allows you to pass additional arguments to the function by adding extra arguments to the call, in the same way that std::thread does.
Also for moving use **std::ref**

std::future and std::async

main ×

```cpp
#include <iostream>
#include <future>

int func(void){
    // do someting
    return 0;
}

int main(){
    std::future<int> fut1 = std::async(func, std::launch::deferred);
    std::future<int> fut2 = std::async(func, std::launch::async);
    std::future<int> fut3 = std::async(func); // Uses default option (both)
    fut1.wait();
    int x = fut2.get();
    int y = fut1.get();
    int z = fut3.get();
    // do something
}
```

The **std::async** can be run in three ways :

1. **std::launch::async** → function must run on its own thread.

2. **std::launch::deferred** → function ran in same thread once **wait()** or **get()** is called.

3. **std::launch::deferred | std::launch::async** → Implementation may choose (default).

std::packaged_task

main ×

```cpp
#include <iostream>
#include <thread>
#include <future>

int main() {
    std::packaged_task<int(int, int)> task([](int a, int b) {
        return a + b;
    });

    std::future<int> result = task.get_future();

    std::thread t(std::move(task), 10, 20);
    t.detach();

    std::cout << "The result is: " << result.get() << std::endl;

    return 0;
}
```

1. The **std::packaged_task<>** is used to make "*task bits*" as I personally like to say them.

2. These "*task bits*" have properties of being fired at any time, with our arguments (like function objects) and beign able to move across threads.

3. These are mainly used to have a good control over what we need to do (task) and how are we going to do it (on the same thread, on another thread, using async, etc.)

4. These "*task bits*" can be used later to make thread pools, which are just queue of tasks used by schedulers in say OS.

5. **std::packaged_task<>** ties a future to a function or callable object. When the **std::packaged_task<>** object is invoked, it calls the associated function or callable object and makes the future ready, with the return value stored as the associated data.

6. The **std::packaged_task<>** itself becomes a callable object.

std::promise

main ×

```cpp
#include <iostream>
#include <thread>
#include <future>
#include <string>

void fetchData(std::promise<std::string>&& p) {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    std::string data = "Success: Data from Database";
    p.set_value(data);
}

int main() {
    std::promise<std::string> myPromise;
    std::future<std::string> myFuture = myPromise.get_future();
    std::thread worker(fetchData, std::move(myPromise));

    std::cout << "Waiting for the worker thread..." << std::endl;

    std::string result = myFuture.get();
    std::cout << "Received: " << result << std::endl;

    worker.join();
    return 0;
}
```

1. **std::promise** along with **std::future** act as two ends of communication pipes.

2. **std::promise<T>** provides a means of setting a value (of type T), which can later be read through an associated **std::future<T>** object. A std::promise/ std::future pair would provide one possible mechanism for this kind of facility; the waiting thread could block on the future, while the thread providing the data could use the promise half of the pairing to set the associated value and make the future ready.

3. We can get the **std::future** object using **get_future()** and set the promise using **set_value()**.

4. We can also do exception handling using **set_exception()** function.

5. **std::packaged_task** and **std::promise** also stores exception in future object on destruction, thus preventing forever wait. (**Broken promise exception**)

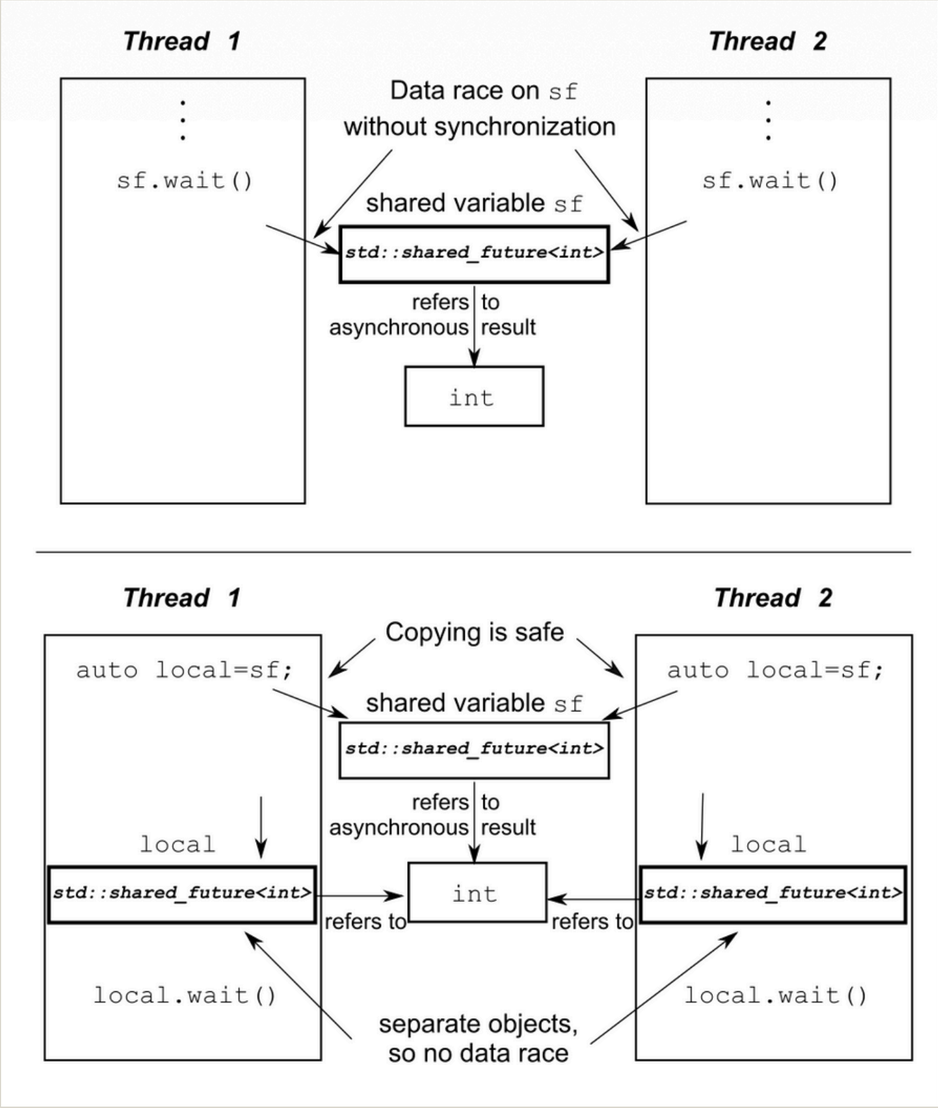
Why not use std::async then? With **std::async**, the value is set only when the function returns. With **std::promise**, you can fulfill the result whenever you want, even in the middle of a long-running function, and then continue doing other work in that same thread.

Confused?? Me too :)

std::future v/s **std::async** v/s **std::packaged_task** v/s **std::promise**

Stores and waits on result from callables	Launches async tasks in bg, returns future	Class which stores “task bits” which can be invoked and moved anytime and anywhere	Other end of two way communication with future
No one runs this. It stores.	Ran by runtime	Manually run by us/consumer thread	Value set using set_value()
Return value is result of get()	Returns future object	Provides future object	Provides future object
Use as the "read-only" handle. It is the object you pass to the part of your code that needs to wait for or check if the result is ready.	Use when you have a specific function (like a heavy math calculation or data sorting) that you want to run in the background immediately without managing the thread yourself.	Use when building a Thread Pool. You wrap various functions into tasks and store them in a queue; a worker thread can then pull and execute them at a later time.	Use to bridge say network events. It allows you to "set" a result from inside writer thread which can then be used by reader thread for transfer of packets.

Waiting from multiple threads

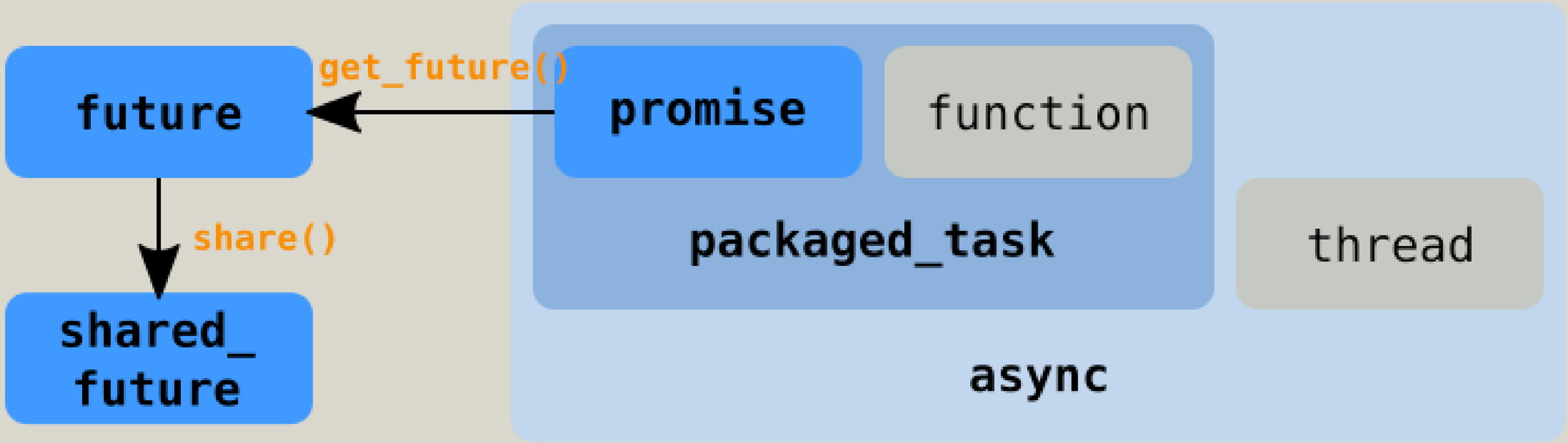


1. The problem is : std::future models unique ownership of the asynchronous result, and the one-shot nature of get() makes concurrent access pointless since only one thread can retrieve the value, because after the first call to get() there's no value left to retrieve

2. This type of facility is thus provided by **std::shared_future**. Whereas std::future is only moveable, so ownership can be transferred between instances, but only one instance refers to a particular asynchronous result at a time, **std::shared_future** instances are copyable, so you can have multiple objects referring to the same associated state.

3. Copying is safe meaning you can make multiple copies of **shared_future** within the same thread as well.

std::future, std::async, std::promise & std::packaged_task



Back to this image now :)

All the things were taken from...

- C++ Concurrency in Action by Anthony Williams :

1. Chapter 4 : Synchronizing concurrent operations

Next session...

- C++ Memory Model and std::atomic - Part 1

Thank You